Q1. Customer Churn Prediction using Logistic Regression

```
# Import required libraries
from datasets import load_dataset
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report,
    roc_curve, roc_auc_score
)
import matplotlib.pyplot as plt

# Load dataset
dataset = load_dataset("scikit-learn/churn-prediction")
df = dataset["train"].to_pandas()

# Select predictors and target
X = df[["tenure", "MonthlyCharges"]]
y = df["Churn"].map({"No": 0, "Yes": 1})

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Fit Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Confusion Matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Performance Metrics
print("\nPerformance Metrics:")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ROC Curve and AUC
y_prob = model.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

print("AUC:", auc)

plt.plot(fpr, tpr, label=f"ROC Curve (AUC={auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Customer Churn")
plt.legend()
plt.show()
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
```

README.md: 100% 1.21k/1.21k [00:00<00:00, 31.7kB/s]

dataset.csv: 100% 970k/970k [00:00<00:00, 11.4MB/s]

Generating train split: 100% 7043/7043 [00:00<00:00, 26335.20 examples/s]

```
Confusion Matrix:
[[931 104]
 [215 159]]

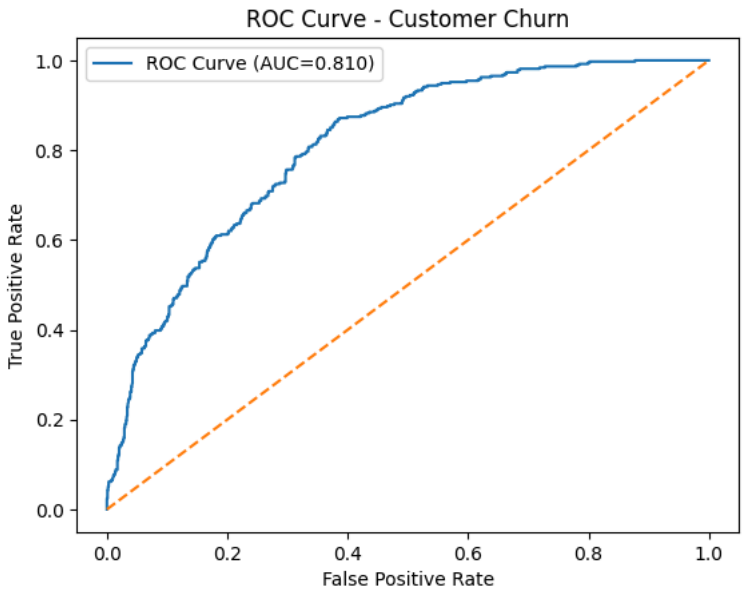
Performance Metrics:
Accuracy: 0.7735982966643009
Precision: 0.6045627376425855
Recall: 0.42513368983957217
F1 Score: 0.49921507064364207

Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.90      0.85      1035
           1       0.60      0.43      0.50       374

    accuracy                           0.77      1409
   macro avg       0.71      0.66      0.68      1409
weighted avg       0.76      0.77      0.76      1409

AUC: 0.8100855098297552
```



Q2. Penguin Species Classification using Logistic Regression

```
# Import required libraries
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report,
    roc_curve, roc_auc_score
)

# Load dataset
df = sns.load_dataset("penguins")

# Select Adelie and Gentoo species
df = df[df["species"].isin(["Adelie", "Gentoo"])].copy()
```

```
# Select predictors
features = [
    "bill_length_mm", "bill_depth_mm",
    "flipper_length_mm", "body_mass_g"
]

df = df[features + ["species"]].dropna()

X = df[features]
y = (df["species"] == "Gentoo").astype(int)

print("Dataset Shape:", df.shape)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize and fit Logistic Regression
modelb = Pipeline([
    ("scaler", StandardScaler()),
    ("logistic", LogisticRegression(max_iter=1000))
])

modelb.fit(X_train, y_train)

# Make predictions
y_pred = modelb.predict(X_test)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Performance Metrics
print("\nPerformance Metrics:")
print("--------------------")
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ROC Curve and AUC
y_prob = modelb.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

print("AUC:", auc)

plt.plot(fpr, tpr, label=f"ROC Curve (AUC={auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Penguin Species Classification")
plt.legend()
plt.show()
```

```
Dataset Shape: (274, 5)

Confusion Matrix:
[[30  0]
 [ 0 25]]

Performance Metrics:
--------------------
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        30
           1       1.00      1.00      1.00        25

    accuracy                           1.00        55
   macro avg       1.00      1.00      1.00        55
weighted avg       1.00      1.00      1.00        55

AUC: 1.0
```

